

LUXURY_ADJECTIVES = [

FULL LIST GOES HERE

Example entries:

"luminous", "opulent", "silken", "refined", "radiant", "sumptuous",

"velvet-soft", "elegant", "polished", "prestige-level",

]

SENSORY_VERBS = [

FULL LIST GOES HERE

Example entries:

"glides", "melts", "envelops", "illuminates", "softens", "smooths",

"cushions", "wraps", "diffuses", "enhances",

]

CONFIDENCE_PHRASES = [

FULL LIST GOES HERE

Example entries:

"own your glow", "step into your power", "move with intention",

"radiate confidence", "command the room", "lead with elegance",

]

EDITORIAL_FRAMES = [

FULL LIST GOES HERE

Example entries:

"Designed to {adj} elevate your {mood} presence.",

"A {sensory} experience crafted for {occasion} moments.",

"Where {benefit} meets {texture} luxury.",

"A modern expression of {persona} energy.",

]

```
PERSONA_CONFIG = {  
  "MVQueen Core": {  
    "tone": "luxury-confident-soft-feminine",  
    "emotional_axes": ["radiance", "self-belief", "soft power"],  
    "seo_bias": ["luxury skincare", "modern beauty", "elevated essentials"],  
    "cta_style": "soft imperative",  
  },  
  "MISS.QUEEN": {  
    "tone": "playful-glam-feminine",  
    "emotional_axes": ["fun", "glow", "self-expression"],  
    "seo_bias": ["soft glam", "night-out looks"],  
    "cta_style": "playful invitation",  
  },  
  "Victoria Soft Power": {  
    "tone": "sensual-soft-power",  
    "emotional_axes": ["allure", "confidence", "intimacy"],  
    "seo_bias": ["lingerie-inspired beauty", "soft power dressing"],  
    "cta_style": "whispered persuasion",  
  },  
  "Sephora Precision": {  
    "tone": "clinical-precise-modern",  
    "emotional_axes": ["results", "clarity", "control"],  
    "seo_bias": ["clinical skincare", "active ingredients"],  
    "cta_style": "clear directive",  
  },  
  "Runway Modern": {
```

```
    "tone": "editorial-bold-runway",
    "emotional_axes": ["impact", "presence", "statement"],
    "seo_bias": ["runway looks", "editorial fashion"],
    "cta_style": "commanding",
  },
  "Clinical Chic": {
    "tone": "minimal-clinical-luxe",
    "emotional_axes": ["purity", "refinement", "assurance"],
    "seo_bias": ["dermatologist-inspired", "clean formulas"],
    "cta_style": "assured guidance",
  },
  "Soft Luxury": {
    "tone": "whisper-soft-luxury",
    "emotional_axes": ["comfort", "ease", "quiet confidence"],
    "seo_bias": ["everyday luxury", "soft textures"],
    "cta_style": "gentle suggestion",
  },
  "Glow Maestro": {
    "tone": "radiant-glow-obsessed",
    "emotional_axes": ["luminosity", "energy", "vitality"],
    "seo_bias": ["glowing skin", "radiant finish"],
    "cta_style": "hype but refined",
  },
  "Elegant Authority": {
    "tone": "refined-commanding-luxe",
    "emotional_axes": ["leadership", "presence", "composure"],
```

```

        "seo_bias": ["power dressing", "boardroom beauty"],
        "cta_style": "decisive invitation",
    },
    "Street Luxe Femme": {
        "tone": "street-modern-luxe",
        "emotional_axes": ["edge", "self-definition", "movement"],
        "seo_bias": ["street style", "city-ready looks"],
        "cta_style": "cool directive",
    },
}
"""

```

Shopify Authentication — MVQueen Engine

Handles API headers and token access.

```

"""

```

```

From mvqueen_engine.config import SHOPIFY_ACCESS_TOKEN

```

```

Def get_headers():

```

```

    Return {
        "X-Shopify-Access-Token": SHOPIFY_ACCESS_TOKEN,
        "Content-Type": "application/json"
    }
"""

```

Session Manager — MVQueen Engine

Handles Shopify API session URLs and pagination.

```

"""

```

Import requests

From mvqueen_engine.shopify_api.auth import get_headers

From mvqueen_engine.config import SHOPIFY_STORE_DOMAIN, SHOPIFY_API_VERSION

Def fetch_all_products():

"""Fetch all Shopify products and return handle → product_id map."""

Products = {}

url =

https://{SHOPIFY_STORE_DOMAIN}/admin/api/{SHOPIFY_API_VERSION}/products.json?limit=250

while url:

r = requests.get(url, headers=get_headers())

r.raise_for_status()

data = r.json().get("products", [])

for p in data:

products[p["handle"]] = p["id"]

url = r.links.get("next", {}).get("url")

return products

"""

Product Updater — MVQueen Engine

Handles safe product updates, metafields, and DRY RUN mode.

"""

Import time

Import requests

From mvqueen_engine.shopify_api.auth import get_headers

From mvqueen_engine.shopify_api.session_manager import fetch_all_products

From mvqueen_engine.config import SHOPIFY_STORE_DOMAIN, SHOPIFY_API_VERSION

Def update_products(df, dry_run=True, sleep_seconds=0.4):

 """Update Shopify products safely."""

 Product_map = fetch_all_products()

 For idx, row in df.iterrows():

 Handle = row["Handle"]

 If handle not in product_map:

 Print(f"Skipping missing product: {handle}")

 Continue

 Product_id = product_map[handle]

 Payload = {

 "product": {

 "id": product_id,

 "title": row["Title"],

 "body_html": row["Body (HTML)"],

 "tags": row["Tags"],

```
        "metafields_global_title_tag": row["SEO Title"],
        "metafields_global_description_tag": row["SEO Description"],
    }
}
```

Build metafields

Metafields = []

For col in row.index:

If col.startswith("metafield.custom."):

Value = str(row[col]).strip()

If value:

Key = col.replace("metafield.custom.", "")

Metafields.append({

"namespace": "custom",

"key": key,

"type": "single_line_text_field",

"value": value

})

If metafields:

Payload["product"]["metafields"] = metafields

If dry_run:

Print(f"DRY RUN → Would update {handle}")

Continue

Try:

```
R = requests.put(
```

```
Fhttps://{SHOPIFY_STORE_DOMAIN}/admin/api/{SHOPIFY_API_VERSION}/products/{product_id}.json,
```

```
    Headers=get_headers(),
```

```
    Json=payload
```

```
)
```

```
r.raise_for_status()
```

```
print(f"Updated {handle}")
```

```
except Exception as e:
```

```
    print(f"Failed {handle}: {e}")
```

```
time.sleep(sleep_seconds)
```

```
"""
```

MVQueen Metafield Engine

Generates all metafields used across the MVQueen ecosystem.

```
"""
```

```
From mvqueen_engine.utils.deterministic import deterministic_choice,  
deterministic_multi
```

```
From mvqueen_engine.brand_brain.brand_banks import (
```

```
    BENEFITS, ROUTINE_STEPS, SKIN_PROFILES, TEXTURES, FINISHES,
```

```
    MOODS, OCCASIONS, BUSINESS_TIERS
```

```
)
```

```
Def generate_metafields(handle: str):
```

"""Generate a full metafield dictionary for a product."""

Return {

 "metafield.custom.key_benefits": ", ".join(

 Deterministic_multi(BENEFITS, handle, count=4)

),

 "metafield.custom.routine_step": deterministic_choice(ROUTINE_STEPS, handle, 0),

 "metafield.custom.who_its_for": deterministic_choice(SKIN_PROFILES, handle, 1),

 "metafield.custom.results": deterministic_choice(BENEFITS, handle, 2),

 "metafield.custom.texture": deterministic_choice(TEXTURES, handle, 3),

 "metafield.custom.finish": deterministic_choice(FINISHES, handle, 4),

 "metafield.custom.mood": deterministic_choice(MOODS, handle, 5),

 "metafield.custom.occasion": deterministic_choice(OCCASIONS, handle, 6),

 "metafield.custom.business_tier": deterministic_choice(BUSINESS_TIERS, handle, 7),

 # SEO metafields

 "metafield.custom.search_priority": "true", "metafield.custom.editorial_ready":
 "true",

 }

"""

MVQueen Metafield Engine

Generates all metafields used across the MVQueen ecosystem.

"""

```
From mvqueen_engine.utils.deterministic import deterministic_choice,  
deterministic_multi
```

```
From mvqueen_engine.brand_brain.brand_banks import (  
    BENEFITS, ROUTINE_STEPS, SKIN_PROFILES, TEXTURES, FINISHES,  
    MOODS, OCCASIONS, BUSINESS_TIERS  
)
```

```
Def generate_metafields(handle: str):
```

```
    """Generate a full metafield dictionary for a product."""
```

```
    Return {
```

```
        "metafield.custom.key_benefits": ", ".join(  
            Deterministic_multi(BENEFITS, handle, count=4)  
        ),
```

```
        "metafield.custom.routine_step": deterministic_choice(ROUTINE_STEPS, handle, 0),
```

```
        "metafield.custom.who_its_for": deterministic_choice(SKIN_PROFILES, handle, 1),
```

```
        "metafield.custom.results": deterministic_choice(BENEFITS, handle, 2),
```

```
        "metafield.custom.texture": deterministic_choice(TEXTURES, handle, 3),
```

```
        "metafield.custom.finish": deterministic_choice(FINISHES, handle, 4),
```

```
        "metafield.custom.mood": deterministic_choice(MOODS, handle, 5),
```

```
        "metafield.custom.occasion": deterministic_choice(OCCASIONS, handle, 6),
```

```
        "metafield.custom.business_tier": deterministic_choice(BUSINESS_TIERS, handle, 7),
```

```
# SEO metafields

“metafield.custom.search_priority”: “true”,

“metafield.custom.editorial_ready”: “true”,

}

“””
```

Shopify Authentication — MVQueen Engine

Handles API headers and token access.

```
“””
```

```
From mvqueen_engine.config import SHOPIFY_ACCESS_TOKEN
```

```
Def get_headers():
```

```
    Return {

        “X-Shopify-Access-Token”: SHOPIFY_ACCESS_TOKEN,

        “Content-Type”: “application/json”

    }

“””
```

Session Manager — MVQueen Engine

Handles Shopify API session URLs and pagination.

```
“””
```

```
Import requests
```

```
From mvqueen_engine.shopify_api.auth import get_headers
```

```
From mvqueen_engine.config import SHOPIFY_STORE_DOMAIN, SHOPIFY_API_VERSION
```

```
Def fetch_all_products():
```

"""Fetch all Shopify products and return handle → product_id map."""

Products = {}

url =

https://{SHOPIFY_STORE_DOMAIN}/admin/api/{SHOPIFY_API_VERSION}/products.json?limit=250

while url:

r = requests.get(url, headers=get_headers())

r.raise_for_status()

data = r.json().get("products", [])

for p in data:

products[p["handle"]] = p["id"]

url = r.links.get("next", {}).get("url")

return products

"""

Shopify Authentication — MVQueen Engine

Handles API headers and token access.

"""

From mvqueen_engine.config import SHOPIFY_ACCESS_TOKEN

Def get_headers():

Return {

"X-Shopify-Access-Token": SHOPIFY_ACCESS_TOKEN,

```
    "Content-Type": "application/json"
```

```
}
```

```
"""
```

Session Manager — MVQueen Engine

Handles Shopify API session URLs and pagination.

```
"""
```

Import requests

```
From mvqueen_engine.shopify_api.auth import get_headers
```

```
From mvqueen_engine.config import SHOPIFY_STORE_DOMAIN, SHOPIFY_API_VERSION
```

```
Def fetch_all_products():
```

```
    """Fetch all Shopify products and return handle → product_id map."""
```

```
    Products = {}
```

```
    url =
```

```
https://{SHOPIFY\_STORE\_DOMAIN}/admin/api/{SHOPIFY\_API\_VERSION}/products.json?limit=250
```

```
    while url:
```

```
        r = requests.get(url, headers=get_headers())
```

```
        r.raise_for_status()
```

```
        data = r.json().get("products", [])
```

```
        for p in data:
```

```
            products[p["handle"]] = p["id"]
```

```
        url = r.links.get("next", {}).get("url")
```

```
    return products
```

```
"""
```

Product Updater — MVQueen Engine

Handles safe product updates, metafields, and DRY RUN mode.

```
"""
```

Import time

Import requests

```
From mvqueen_engine.shopify_api.auth import get_headers
```

```
From mvqueen_engine.shopify_api.session_manager import fetch_all_products
```

```
From mvqueen_engine.config import SHOPIFY_STORE_DOMAIN, SHOPIFY_API_VERSION
```

```
Def update_products(df, dry_run=True, sleep_seconds=0.4):
```

```
    """Update Shopify products safely."""
```

```
    Product_map = fetch_all_products()
```

```
    For idx, row in df.iterrows():
```

```
        Handle = row["Handle"]
```

```
        If handle not in product_map:
```

```
            Print(f"Skipping missing product: {handle}")
```

```
            Continue
```

```
        Product_id = product_map[handle]
```

```
Payload = {  
    "product": {  
        "id": product_id,  
        "title": row["Title"],  
        "body_html": row["Body (HTML)"],  
        "tags": row["Tags"],  
        "metafields_global_title_tag": row["SEO Title"],  
        "metafields_global_description_tag": row["SEO Description"],  
    }  
}
```

```
# Build metafields
```

```
Metafields = []
```

```
For col in row.index:
```

```
    If col.startswith("metafield.custom."):
```

```
        Value = str(row[col]).strip()
```

```
        If value:
```

```
            Key = col.replace("metafield.custom.", "")
```

```
            Metafields.append({
```

```
                "namespace": "custom",
```

```
                "key": key,
```

```
                "type": "single_line_text_field",
```

```
                "value": value
```

```
            })
```

```
If metafields:
```

```
Payload["product"]["metafields"] = metafields
```

```
If dry_run:
```

```
    Print(f"DRY RUN → Would update {handle}")
```

```
    Continue
```

```
Try:
```

```
    R = requests.put(
```

```
        https://{SHOPIFY\_STORE\_DOMAIN}/admin/api/{SHOPIFY\_API\_VERSION}/products/{product\_id}.json,
```

```
        Headers=get_headers(),
```

```
        Json=payload
```

```
    )
```

```
    r.raise_for_status()
```

```
    print(f"Updated {handle}")
```

```
except Exception as e:
```

```
    print(f"Failed {handle}: {e}")
```

```
time.sleep(sleep_seconds)
```